Warmup Week 7 - Utilities

Rewritten for Mathematica 14.2 which (finally!) contains some game theory functions...

Problem 1

1.

```
In[•]:= game1 = MatrixGame[
          {{{5, 5}, {10, 4}}, {{4, 10}, {2, 2}}},
          GameActionLabels → {{"Good", "Mediocre"}, {"Good", "Mediocre"}}
         ]
```

Out[•]=

MatrixGame [Number of Players: 2
Number of Actions : {2, 2}]

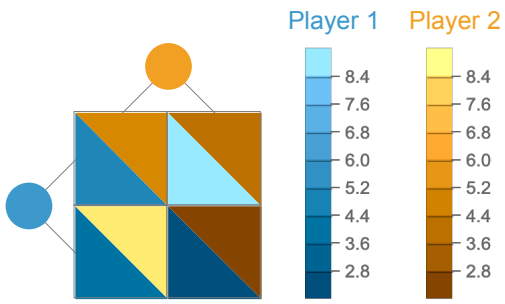
```
In[•]:= game1["Dataset"]
```

Out[•]=

	Good		Mediocre	
Good	5	5	10	4
Mediocre	4	10	2	2

```
In[•]:= MatrixGamePlot[game1, PlotLayout → "SplitSquare"]
```

Out[•]=



2. Yes, never go to the mediocre one

3.

```
In[•]:= MatrixGamePayoff[game1, {{0.5, 0.5}, {0.9, 0.1}}]
```

Out[•]=

```
{4.65, 7.05}
```

To calculate this explicitly: The probabilities for the outcomes are

Explicitly calculated, the expected payoff for each player thus is

In[•]:=
```
π1 = .45 * 5 + .05 * 10 + .45 * 4 + .05 * 2
π2 = .45 * 5 + .05 * 4 + .45 * 10 + .05 * 2
```

Out[•]=
```
4.65
```

Out[•]=
```
7.05
```

Problem 2

1. payoff in $k. The actions in the matrix are in order of increasing claims: to

In[•]:=
```
A = ((2 4 4 4; 0 3 5 5; 0 1 4 6; 0 1 2 5)); B = ((2 0 0 0; 4 3 1 1; 4 5 4 2; 4 5 6 5));
```

In[•]:=
```
B == Transpose[A]
```

Out[•]=
```
True
```

Note: *Please make sure that you only use one Matrix notation (either two separate matrices or bi-matrix) in the lab. Which one you choose doesn't matter.*

In[•]:=
```
game2 = MatrixGame[Transpose[{A, B}, 2],
  GameActionLabels → ConstantArray[Table["$" <> ToString[i] <> "k", {i, 4}], 2]]
```

Out[•]=

MatrixGame [Number of Players: 2
Number of Actions : {4, 4}]

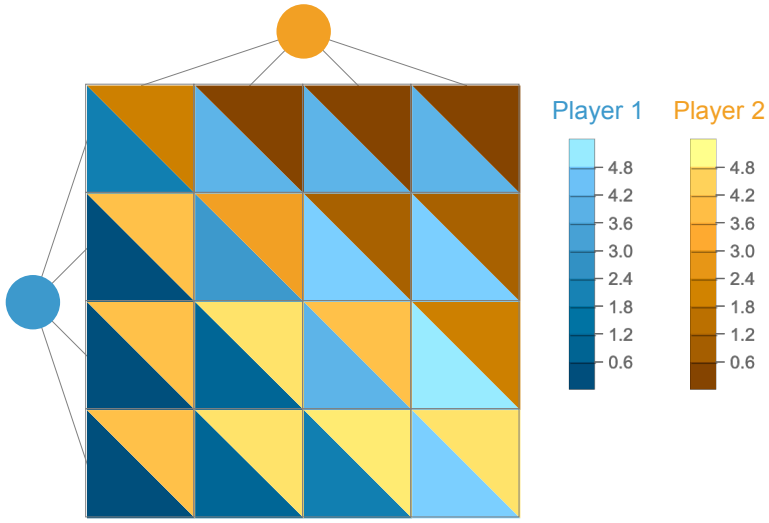
In[•]:=
```
game2["Dataset"]
```

Out[•]=

	$1k		$2k		$3k		$4k	
$1k	2	2	4	0	4	0	4	0
$2k	0	4	3	3	5	1	5	1
$3k	0	4	1	5	4	4	6	2
$4k	0	4	1	5	2	6	5	5

In[•]:=
```
MatrixGamePlot[game2, PlotLayout → "SplitSquare"]
```

Out[•]=



Problem 3

1. There is an external influence that is not under the control of the two players. This external random influence influences the payoffs . If we do not even know anything about the event distribution, we cannot even formulate this as an expected value problem. We could also formulate this as a 3-player problem. Except, the lion's got nothing to win!

2. If we know the probability of the outcomes we can calculate the expected utility as in Problem 1:

In[•]:=
```
payoffMatrix = Dataset[
  <|
   "F" → <| "F" → 0, "P" → .75 * 100 + .25 * -100 |>,
   "P" → <| "F" → .75 * -100 + .25 * 100, "P" → 0 |>
   |>
 ]
```

Out[•]=

	F	P
F	0	50.0
P	−50.0	0

pretty boring game to play!

Problem 4

We have now completely removed the collaboration component from the game and only leave the competition component in place. This can be expressed thorugh a rewritten payoff matrix.

In[•]:=
```
game1 = MatrixGame[
  {{{1, -1}, {0, 0}}, {{0, 0}, {-1, 1}}},
  GameActionLabels → {{"B", "F"}, {"B", "F"}}
 ]
```

Out[•]=

MatrixGame [Number of Players: 2
Number of Actions : {2, 2}]

In[•]:=
```
game1["Dataset"]
```

Out[•]=

	B		F	
B	1	−1	0	0
F	0	0	−1	1

This game has a single (pure) Nash equilibirum, Player 1 can only win by playing “B” and Plyer 2 can only win by playing “F”, so that’s

In[•]:=
```
FindMatrixGameStrategies[game1, "Mixed"]
```

Out[•]=
```
{}
```

In[•]:=
```
FindMatrixGameStrategies[game1, "Pure"]
```

Out[•]=
```
{{{1, 0}, {0, 1}}}
```

In consequence, both player can expect to go empty-handed.

In[•]:=
```
MatrixGamePayoff[game1, First[%]]
```

Out[•]=
```
{0, 0}
```